

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»
Навчально-науковий “Фізико-технічний інститут”

Лабораторна робота №2
з дисципліни «Технологія блокчейн та розподілені системи»

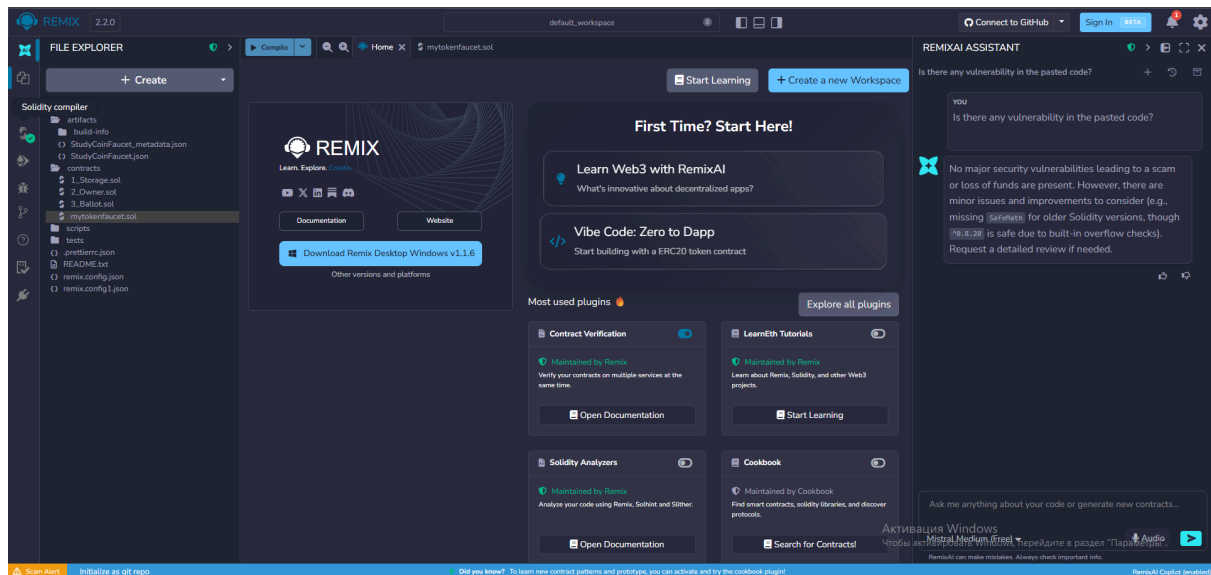
Виконав:
Група:

Петренко Олександр
ФІ-51мн

Завдання: Розгортання та запуск обраної анонімної валюти, протоколювання майнінгу, пошук слідів деанонімізації; розгортання та запуск обраного смарт-контракту, підвищення ефективності роботи смарт-контракту з точки зору витрати гасу; Варіант: розробка власного смарт-контракту.

Хід роботи

1. Заходимо в Remix IDE:



2. Створюємо смарт-контракт mytokenfaucet.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

contract StudyCoinFaucet {
    string public name = "MyTestToken";
    string public symbol = "MTT";
    uint8 public decimals = 18;

    uint256 public totalSupply;
    address public owner;

    uint256 public dripAmount = 100 * 10 ** 18;
    uint256 public cooldownTime = 1 minutes;

    mapping(address => uint256) public balanceOf;
    mapping(address => uint256) public lastRequestTime;
    mapping(address => mapping(address => uint256)) public allowance;
```

```

    event Transfer(address indexed from, address indexed to, uint256
value);
    event Approval(address indexed tokenOwner, address indexed spender,
uint256 value);
    event TokensRequested(address indexed user, uint256 amount);
    event FaucetRefilled(uint256 amount);

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can call this
function");
        _;
    }

    constructor() {
        owner = msg.sender;

        uint256 initialSupply = 1_000_000 * 10 ** uint256(decimals);

        totalSupply = initialSupply;
        balanceOf[address(this)] = initialSupply;

        emit Transfer(address(0), address(this), initialSupply);
    }

    function requestTokens() external {
        require(
            block.timestamp >= lastRequestTime[msg.sender] +
cooldownTime,
            "Please wait before requesting tokens again"
        );

        require(
            balanceOf[address(this)] >= dripAmount,
            "Faucet does not have enough tokens"
        );

        lastRequestTime[msg.sender] = block.timestamp;

        balanceOf[address(this)] -= dripAmount;
        balanceOf[msg.sender] += dripAmount;

        emit Transfer(address(this), msg.sender, dripAmount);
        emit TokensRequested(msg.sender, dripAmount);
    }

```

```

    }

    function transfer(address to, uint256 amount) external returns
(bool) {
        require(to != address(0), "Invalid address");
        require(balanceOf[msg.sender] >= amount, "Not enough tokens");

        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;

        emit Transfer(msg.sender, to, amount);

        return true;
    }

    function approve(address spender, uint256 amount) external returns
(bool) {
        require(spender != address(0), "Invalid address");

        allowance[msg.sender][spender] = amount;

        emit Approval(msg.sender, spender, amount);

        return true;
    }

    function transferFrom(
        address from,
        address to,
        uint256 amount
    ) external returns (bool) {
        require(to != address(0), "Invalid address");
        require(balanceOf[from] >= amount, "Not enough tokens");
        require(allowance[from][msg.sender] >= amount, "Allowance
exceeded");

        allowance[from][msg.sender] -= amount;
        balanceOf[from] -= amount;
        balanceOf[to] += amount;

        emit Transfer(from, to, amount);

        return true;
    }

```

```

    }

    function mintToFaucet(uint256 amount) external onlyOwner {
        uint256 tokenAmount = amount * 10 ** uint256(decimals);

        totalSupply += tokenAmount;
        balanceOf[address(this)] += tokenAmount;

        emit Transfer(address(0), address(this), tokenAmount);
        emit FaucetRefilled(tokenAmount);
    }

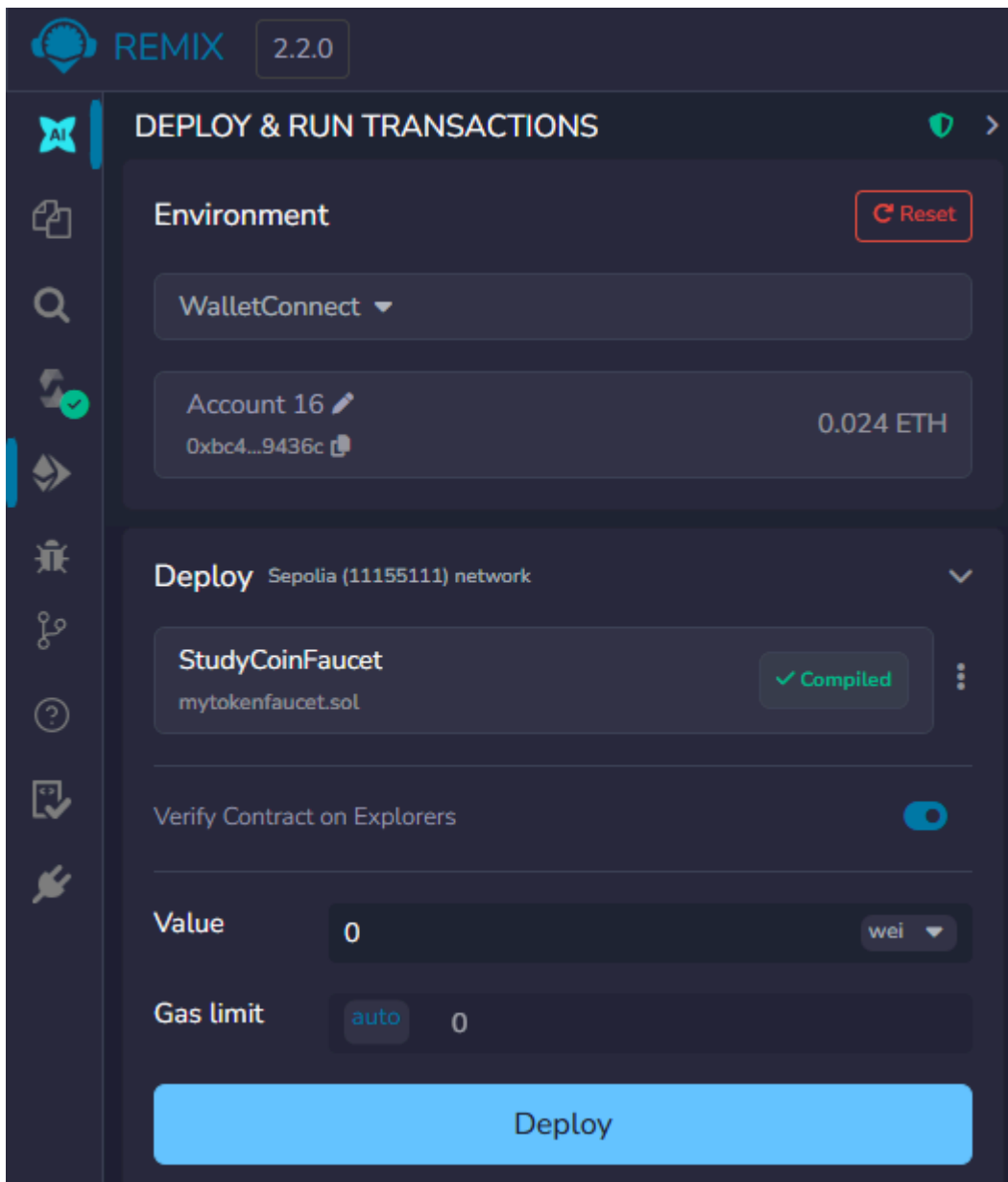
    function setDripAmount(uint256 amount) external onlyOwner {
        dripAmount = amount * 10 ** uint256(decimals);
    }

    function setCooldownTime(uint256 timeInSeconds) external onlyOwner
{
        cooldownTime = timeInSeconds;
    }

    function getFaucetBalance() external view returns (uint256) {
        return balanceOf[address(this)];
    }
}

```

3. Компілюємо контракт і підключаємо Metamask:



4. Deploy:

 Account 1

Развернуть контракт

Этот сайт хочет, чтобы вы развернули контракт

Прогнозируемые изменения

Сеть

S Sepolia

Запрос от

remix.ethereum.org

Комиссия сети

Скорость

Рынок ~24 сек.

Отмена

Подтвердить

 TRANSACTION ACTION

Transfer 1,000,000 to 0xa293e7b11D8c08aD38850aCf4E7e15C839Beb7d4

[This is a Sepolia **Testnet** transaction only]

Transaction Hash:

0x31e4ec1c70ca033eba04a391626f933da09b6b134960a20726b347f9240baf41

Status:

Success

Block:

10891286 1 Block Confirmation

Timestamp:

22 secs ago | May-21-2026 10:47:36 AM +UTC

From:

0xBc42F25Cfc7e5909b88c180DC0F1B3fB6F29436c

To:

[0xa293e7b11d8c08ad38850ac4e7e15c839beb7d4 Created]

Value:

0 ETH

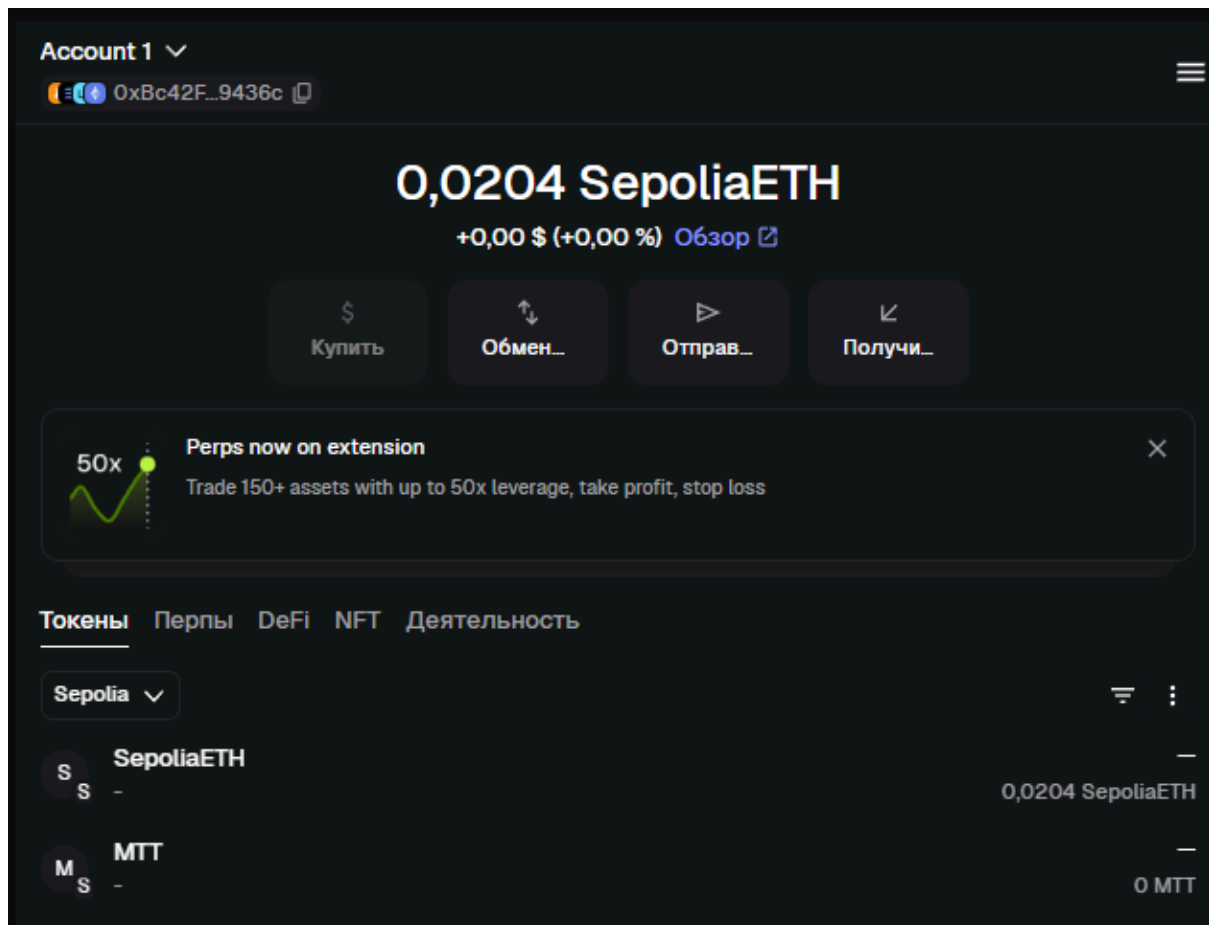
Transaction Fee:

0.004592962588617118 ETH

Gas Price:

2.649650569 Gwei (0.000000002649650569 ETH)

5. Імпорт токена у гаманці:



6. Список функцій контракта:

name - назва токена.

symbol - скорочене позначення токена.

decimals - кількість знаків після коми, дорівнює 18.

totalSupply - загальна кількість випущених tokenів.

owner - адреса власника смарт-контракту.

dripAmount - кількість tokenів, яку faucet видає за один запит.

balanceOf - показує баланс tokenів на конкретній адресі.

allowance - показує, скільки tokenів одна адреса дозволила витратити іншій адресі.

requestTokens - основна функція крана, видає користувачу фіксовану кількість tokenів.

transfer - переказує токени з адреси відправника на іншу адресу.

approve - дозволяє іншій адресі використовувати певну кількість tokenів власника.

transferFrom - переказує токени з однієї адреси на іншу, якщо раніше був наданий дозвіл через approve.

mintToFaucet - створює нові токени та додає їх на баланс faucet, доступна тільки власнику.

setDripAmount - змінює кількість tokenів, яку faucet видає за один запит, доступна тільки власнику.

getFaucetBalance - показує, скільки tokenів залишилося на балансі faucet.

● **symbol**
o:string: MTT

● **owner**
o:address: 0xBc42F25Cfc7e5909b88c180DC0F1B3fB6F29436c

● **name**
o:string: MyTestToken

● **getFaucetBalance**
o:uint256: 99990000000000000000000000000000

● **dripAmount**
o:uint256: 100000000000000000000000000000000

● **cooldownTime**
o:uint256: 60

● **balanceOf** address
0xBc42F25Cfc7e5909b88c180DC0F1B3fB6F29436c
Calldata ⓘ Parameters ⓘ
o:uint256: 100000000000000000000000000000000

✓ [block:10891366 txIndex:35] from: 0xBc4...9436c to: StudyCoinFaucet.mintToFaucet(uint256) 0xa29...eb7d4 value: 0 wei data: 0x541...00096 logs: 2 hash: 0x499...a7330

Висновок

У ході роботи було розроблено та протестовано смарт-контракт StudyCoinFaucet у середовищі Remix. Контракт створює власний токен StudyCoin з позначенням STC та дозволяє користувачам отримувати токени через функцію requestTokens. Також були створені і протестовані інші функції (детальніше в п.7)